

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №7
по курсу «Алгоритмы и структуры данных»
Тема: Динамическое программирование №1
Очередь с приоритетами.
Вариант 20

Выполнила:
Степанова Е.А.
К3141

Проверил:
Афанасьев А.В.

Санкт-Петербург
2024г.

Содержание отчета

Содержание отчета	2
Задачи по варианту	3
<u>Задача №1. Обмен монет [N баллов]</u>	
<u>Задача №2. Высота дерева [N баллов]</u>	6
<u>Задача №4. Построение пирамиды [N баллов]</u>	19
Вывод	23

Задачи по варианту

Задача №1. Обмен монет [N баллов]

1 задача. Обмен монет

Как мы уже поняли из лекции, не всегда "жадное" решение задачи на обмен монет работает корректно для разных наборов номиналов монет. Например, если доступны номиналы 1, 3 и 4, жадный алгоритм поменяет 6 центов, используя три монеты (4 + 1 + 1), в то время как его можно изменить, используя всего две монеты (3 + 3). Теперь ваша цель - применить динамическое программирование для решения задачи про обмен монет для разных номиналов.

- **Формат ввода / входного файла (input.txt).** Целое число $money$ ($1 \leq money \leq 10^3$). Набор монет: количество возможных монет k и сам набор $coins = \{coin_1, \dots, coin_k\}$. $1 \leq k \leq 100$, $1 \leq coin_i \leq 10^3$. Проверку можно сделать на наборе {1, 3, 4}. Формат ввода: первая строка содержит через пробел $money$ и k ; вторая - $coin_1 coin_2 \dots coin_k$.

– **Вариация 2:** Количество монет в кассе ограничено. Для каждой монеты из набора $coins = \{coin_1, \dots, coin_k\}$ есть соответствующее целое число - количество монет в кассе данного номинала $c = \{c_1, \dots, c_k\}$. Если они закончились, то выдать данную монету невозможно.

- **Формат вывода / выходного файла (output.txt).** Вывести одно число – минимальное количество необходимых монет для размена $money$ доступным набором монет $coins$.
- Ограничение по времени. 1 сек.
- Примеры:

input.txt	output.txt	input.txt	output.txt
2 3	2	34 3	9
1 3 4		1 3 4	

```
from lab_7.utils import read_f, write_f

def change_coins(money, coins):
    table = [float('inf') for _ in range(money + 1)]
    table[0] = 0
    for m in range(1, money + 1):
        for c in coins:
            if c <= m:
                table[m] = min(table[m], 1 + table[m - c])
    print(table)
```

```

return table[money]

if __name__ == "__main__":
    data, vector = read_f(1)
    data = list(map(int, data.split()))
    money, n = data[0], data[1]
    coins = list(map(int, vector.split()))
    result = change_coins(money, coins)
    print(result)
    write_f(1, result)

```

Считываем данные с помощью `utils`, задаем переменную, отвечающую за общую сумму(которую необходимо собрать минимальным количеством монет), количество видов монет и сам список монет.

Для решения ввожу функцию **`change_coins(money, coins)`**, внутри нее ввожу список, который изначально устанавливаю как везде бесконечность. Устанавливаю первый элемент 0.

Далее прохожусь по всему отрезку от единицы, до нашей суммы.

Задаю цикл, который проходится по всем значениям списка `coins`.

Если номинал меньше чем число, которое входит в отрезок от 1 до самого числа, устанавливаю, что таблица от индекса `m`(от вот этого числа) - это минимум из значения `table[m]` или минимум из предыдущего значения таблицы + 1.

Пусть $change(money)$ — это минимальное количество монет номиналами 1, 3 и 4, которые нужны для сдачи с $money$, а $(c_1, \dots, c_k)$ — оптимальная сдача с $money$. В таком случае

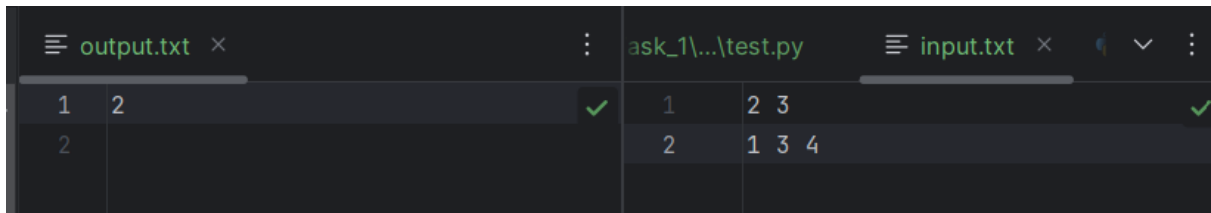
$$c_1 + \dots + c_k = money.$$

Тогда

$$c_1 + \dots + c_{k-1} = money - c_k.$$

Следовательно, $change(money - c_k) = k - 1$. Таким образом, для решения задачи при $money$ достаточно решить её при $money - c_k$ и добавить единицу.

Возвращаю значение `table[money]`.



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000848759536314e-06	0
Пример из задачи	1.0000000000000838e-05	0
Верхняя граница диапазона значений входных данных из текста задачи	1.0000000000000838e-05	0

Задача №3. Редакционное расстояние [N баллов]

Редакционное расстояние между двумя строками – это минимальное количество операций (вставки, удаления и замены символов) для преобразования одной строки в другую. Это мера сходства двух строк. У редакционного расстояния есть применения, например, в вычислительной биологии, обработке текстов на естественном языке и проверке орфографии. Ваша цель в этой задаче – вычислить расстояние редактирования между двумя строками.

- **Формат ввода / входного файла (input.txt).** Каждая из двух строк ввода содержит строку, состоящую из строчных латинских букв. Длина обеих строк - от 1 до 5000.
- **Формат вывода / выходного файла (output.txt).** Выведите расстояние редактирования между заданными двумя строками.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Примеры:

input.txt	output.txt	input.txt	output.txt	input.txt	output.txt
ab	0	short	3	editing	5
ab		ports		distance	

```

from lab_7.utils import read_f, write_f

def edit_dist(s1, s2):
    m, n = len(s1), len(s2)
    prev = list(range(n + 1))

    # Rest of the rows
    for i in range(1, m + 1):
        curr = [i] # j = 0
        for j in range(1, n + 1):
            if s1[i - 1] == s2[j - 1]:
                curr.append(prev[j - 1])
            else:
                curr.append(1 + min(curr[-1],
prev[j], prev[j - 1]))
        prev = curr
    return prev[n]

if __name__ == "__main__":
    str_1, str_2 = read_f(3)
    result = edit_dist(str_1, str_2)
    write_f(3, result)

```

Код считывает две строки из файла и вычисляет расстояние редактирования между ними. Затем результат записывается в файл.

Функции `read_f` и `write_f` — это функции, которые определены в модуле `lab_7.utils`. Они используются для чтения данных из файла и записи данных в файл соответственно. В данном случае они считывают две строки из файла, а затем записывают результат вычисления расстояния редактирования в тот же файл.

Функция `edit_dist` принимает на вход две строки `s1` и `s2` и возвращает расстояние редактирования между этими строками. Расстояние редактирования — это минимальное количество операций вставки, удаления и замены символов, необходимых для преобразования одной строки в другую.

Алгоритм работает следующим образом:

Инициализируется массив `prev`, который будет хранить историю изменений. Изначально он заполняется нулями.

Для каждой позиции `i` в строке `s1`, начиная с первой, выполняется следующее:

Создаётся новый массив `curr`, который будет содержать текущее состояние изменений. Он инициализируется значением `i`.

Для каждой позиции `j` в строке `s2`, начиная со второй, выполняется сравнение символов в позициях `i - 1` и `j - 1`. Если символы совпадают, то значение в массиве `curr` остаётся прежним. Иначе выполняется поиск минимального значения из трёх возможных вариантов:

Значение в предыдущей позиции массива `curr`. Это соответствует удалению символа из строки `s1`.

Значение в текущей позиции массива `prev`. Это соответствует вставке символа в строку `s1`.

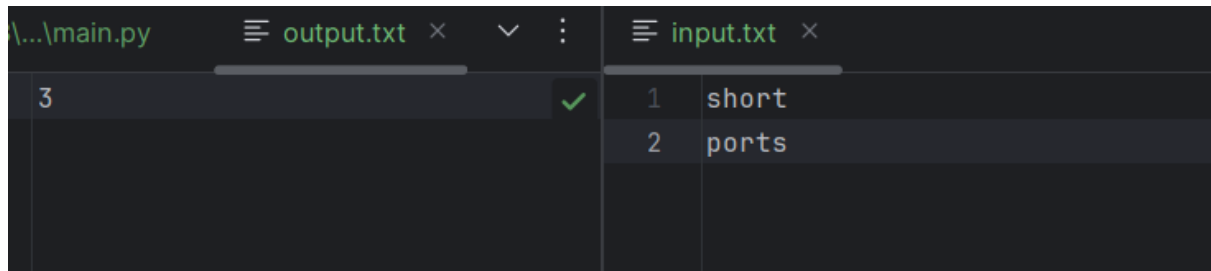
Сумма значений в текущей и предыдущей позициях массива `prev` плюс единица. Это соответствует замене символа в строке `s1`.

Массив `prev` обновляется текущим массивом `curr`.

После завершения цикла возвращается значение из последней позиции массива `prev`, которое соответствует расстоянию редактирования.

В основной программе строки считываются из файла с помощью функции `read_f`, передаются в функцию `edit_dist`, где вычисляется расстояние

редактирования, и записываются обратно в файл с помощью функции `write_f`.



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.110848759536314e-06	56
Пример из задачи	1.2499996955739334e-05	56
Верхняя граница диапазона значений входных данных из текста задачи	1.2499996955739334e-05	56

Задача №4. Наибольшая общая подпоследовательность двух последовательностей [N баллов]

звездочкой ★.

4 задача. Наибольшая общая подпоследовательность двух последовательностей

Вычислить длину самой длинной общей подпоследовательности из двух последовательностей.

Даны две последовательности $A = (a_1, a_2, \dots, a_n)$ и $B = (b_1, b_2, \dots, b_m)$, найти длину их самой длинной общей подпоследовательности, т.е. наибольшее неотрицательное целое число p такое, что существуют индексы $1 \leq i_1 < i_2 < \dots < i_p \leq n$ и $1 \leq j_1 < j_2 < \dots < j_p \leq m$ такие, что $a_{i_1} = b_{j_1}, \dots, a_{i_p} = b_{j_p}$.

- **Формат ввода / входного файла (input.txt).**

- Первая строка: n - длина первой последовательности.
- Вторая строка: $a_1, a_2, \dots, a_n$ через пробел.
- Третья строка: m - длина второй последовательности.

- Четвертая строка: $b_1, b_2, \dots, b_m$ через пробел.

- Ограничения: $1 \leq n, m \leq 100$; $-10^9 < a_i, b_i < 10^9$.

- **Формат вывода / выходного файла (output.txt).** Выведите число p .

- Ограничение по времени. 1 сек.

- Примеры:

input.txt	output.txt	input.txt	output.txt	input.txt	output.txt
3	2	1	0	4	2
2 7 5		7		2 7 8 3	
2		4		4	
2 5		1 2 3 4		5 2 8 7	

- В первом примере одна общая подпоследовательность – (2, 5) длиной 2, во втором примере две последовательности не имеют одинаковых элементов. В третьем примере - длина 2, последовательности – (2, 7) или (2, 8).

```
from lab_7.utils import read_f, write_f

def lcs(A, B, n, m):
    table = [[0 for _ in range(m + 1)] for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if A[i - 1] == B[j - 1]:
                table[i][j] = table[i - 1][j - 1] + 1
```

```

        else:
            table[i][j] = max(table[i - 1][j],
table[i][j - 1])
        return table[n][m]

if __name__ == "__main__":
    data = read_f(4)
    str_1, str_2 = data[1], data[3]
    str_1 = str_1.replace(" ", '')
    str_2 = str_2.replace(" ", '')
    n, m = int(data[0]), int(data[2])
    result = lcs(str_1, str_2, n, m)
    write_f(4, result)

```

В этом коде используются функции `read\_f` и `write\_f`, которые, вероятно, были определены в файле `lab\_7.utils`.

Функция `lcs` находит длину наибольшей общей подпоследовательности (LCS) между двумя строками. Она принимает на вход две строки `A` и `B`, их длины `n` и `m` соответственно и возвращает длину LCS.

В основной программе считываются данные из файла с помощью функции `read\_f`. Затем строки разделяются пробелами заменяются на пустые строки с помощью метода `replace`. После этого вычисляется длина LCS с помощью функции `lcs`, и результат записывается в файл

output.txt			Task_4\...\input.txt		
1	2	✓	1	3	✓
2			2	2 7 5	
			3	2	
			4	2 5	

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000123559536314e-06	0
Пример из задачи	1.0000000000000838e-05	0
Верхняя граница диапазона значений входных данных из текста задачи	1.0000091819822e-05	0

Задача №6. Наибольшая возрастающая подпоследовательность [N баллов]

6 задача. Наибольшая возрастающая подпоследовательность

Дана последовательность, требуется найти ее наибольшую возрастающую подпоследовательность.

- **Формат ввода / входного файла (input.txt).** В первой строке входных данных задано целое число n – длина последовательности ($1 \leq n \leq 300000$). Во второй строке задается сама последовательность. Числа разделяются пробелом. Элементы последовательности – целые числа, не превосходящие по модулю 10^9 .

- Подзадача 1 (полегче). $n \leq 5000$.
- Общая подзадача. $n \leq 300000$.

- **Формат вывода / выходного файла (output.txt).** В первой строке выведите длину наибольшей возрастающей подпоследовательности, а во второй строке выведите через пробел саму наибольшую возрастающую подпоследовательность данной последовательности. Если ответов несколько - выведите любой.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
6	3
3 29 5 5 28 6	3 5 28

```
from lab_7.utils import read_f, write_f
```

```
def get_lis(arr):
```

```

n = len(arr)

# Initialize dp array with 1
dp = [1] * n

# Initialize seq array with index values (to
store previous indices in LIS)
seq = list(range(n))

# Compute dp and seq arrays
for i in range(n):
    for prev in range(i):
        if arr[prev] < arr[i] and 1 + dp[prev] >
dp[i]:
            dp[i] = 1 + dp[prev]
            seq[i] = prev

# Find the index of the last element in the LIS
ans = -1
ans_ind = -1
for i in range(n):
    if dp[i] > ans:
        ans = dp[i]
        ans_ind = i

# Construct the result sequence using seq array
res = []
res.append(arr[ans_ind])
while seq[ans_ind] != ans_ind:
    ans_ind = seq[ans_ind]
    res.append(arr[ans_ind])

# Reverse the result to get the correct order
res.reverse()
return res

if __name__ == "__main__":
    n, data = read_f(6)
    seq = list(map(int, data.split()))
    res = get_lis(seq)

```

```
ans_len = len(res)
write_f(6, *[ans_len, res])
```

output.txt		input.txt	
1	3	1	6
2	[3, 5, 28]	2	3 29 5 5 28 6
3			

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.1399999493733048e-05-05	88
Пример из задачи	1.1399999493733048e-05-05	88
Верхняя граница диапазона значений входных данных из текста задачи	1.1.1399999493733048e-05-05	88

Алгоритм работает следующим образом:

Инициализируется массив `dp` с единицами. Он будет хранить длины LIS для каждого элемента массива `arr`.

Инициализируется массив `seq`, который будет содержать индексы предыдущих элементов в LIS.

Для каждого элемента `i` массива `arr` выполняется перебор всех предыдущих элементов `prev`. Если элемент `prev` меньше элемента `i`, и длина LIS для элемента `prev` плюс один больше текущей длины LIS, то текущая длина LIS устанавливается равной длине LIS элемента `prev`, увеличенной на

один, а индекс предыдущего элемента в LIS — равным индексу элемента prev.

На следующем этапе находится индекс последнего элемента в LIS и его значение добавляется в список res. Затем происходит рекурсивный поиск предыдущего элемента, пока не будет достигнут элемент, который является первым в LIS. Значения всех найденных элементов также добавляются в список res.

Вывод

В данной лабораторной работе мы познакомились с динамическим программированием.